

```

# =====
# INTERNSHIP TASK 6
# Deep Learning Fundamentals & Neural Network Implementation
# =====

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout

print("TensorFlow Version:", tf.__version__)

# =====
# 1. LOAD DATASET
# =====

data = load_breast_cancer()

X = data.data
y = data.target

print("\n===== DATASET =====")
print("Dataset Shape:", X.shape)
print("Target Shape :", y.shape)
print("Number of Features:", X.shape[1])
print("Classes:", np.unique(y))

# =====
# 2. TRAIN-TEST SPLIT
# =====

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\n===== DATA SPLIT =====")
print("Training Data:", X_train.shape)
print("Testing Data :", X_test.shape)

# =====
# 3. FEATURE SCALING
# =====

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

print("\nFeature scaling completed.")

# =====
# 4. BUILD ARTIFICIAL NEURAL NETWORK
# =====

model = Sequential([
    Dense(32, activation="relu", input_shape=(X_train.shape[1],)),
    Dense(16, activation="relu"),
    Dense(1, activation="sigmoid")
])

print("\n===== ANN MODEL =====")
model.summary()

```

```

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

print("\nModel compiled successfully.")

# =====
# 5. COMPILE MODEL
# =====

history = model.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.20,
    verbose=1
)

# =====
# 6. TRAIN MODEL
# =====

print("\n===== MODEL TRAINING =====")

loss, accuracy = model.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\n===== ANN EVALUATION =====")
print("Test Loss      :", round(loss, 4))
print("Test Accuracy:", round(accuracy, 4))

# =====
# 7. EVALUATE MODEL
# =====

predictions = model.predict(X_test, verbose=0)

predicted_classes = (predictions >= 0.5).astype(int).flatten()

# =====
# 8. MAKE PREDICTIONS
# =====

precision = precision_score(
    y_test,
    predicted_classes,
    zero_division=0
)

recall = recall_score(
    y_test,
    predicted_classes,
    zero_division=0
)

f1 = f1_score(
    y_test,
    predicted_classes,
    zero_division=0
)

print("\n===== CLASSIFICATION METRICS =====")
print("Accuracy :", round(accuracy_score(y_test, predicted_classes), 4))
print("Precision:", round(precision, 4))
print("Recall   :", round(recall, 4))
print("F1 Score :", round(f1, 4))

print("\nClassification Report:")
print(
    classification_report(
        y_test,

```

```

        predicted_classes,
        target_names=data.target_names,
        zero_division=0
    )
)

# =====
# 10. CONFUSION MATRIX
# =====

cm = confusion_matrix(
    y_test,
    predicted_classes
)

print("\n===== CONFUSION MATRIX =====")
print(cm)

plt.figure(figsize=(6, 5))
plt.imshow(cm)
plt.title("ANN Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.colorbar()

for i in range(len(cm)):
    for j in range(len(cm)):
        plt.text(
            j,
            i,
            cm[i, j],
            ha="center",
            va="center"
        )

plt.show()

# =====
# 11. TRAINING VS VALIDATION ACCURACY
# =====

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history.history["val_accuracy"],
    label="Validation Accuracy"
)

plt.title("Training vs Validation Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()

# =====
# 12. TRAINING VS VALIDATION LOSS
# =====

plt.figure(figsize=(8, 5))

plt.plot(
    history.history["loss"],
    label="Training Loss"
)

plt.plot(
    history.history["val_loss"],
    label="Validation Loss"
)

plt.title("Training vs Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)

```

```

plt.show()

# =====
# 13. ANN WITH DROPOUT
# =====

print("\n===== ANN WITH DROPOUT =====")

model_dropout = Sequential([
    Dense(32, activation="relu", input_shape=(X_train.shape[1],)),
    Dropout(0.30),
    Dense(16, activation="relu"),
    Dropout(0.20),
    Dense(1, activation="sigmoid")
])

model_dropout.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

history_dropout = model_dropout.fit(
    X_train,
    y_train,
    epochs=30,
    batch_size=32,
    validation_split=0.20,
    verbose=1
)

# =====
# 14. EVALUATE DROPOUT MODEL
# =====

dropout_loss, dropout_accuracy = model_dropout.evaluate(
    X_test,
    y_test,
    verbose=0
)

print("\n===== DROPOUT MODEL RESULTS =====")
print("Test Loss      :", round(dropout_loss, 4))
print("Test Accuracy:", round(dropout_accuracy, 4))

# =====
# 15. COMPARE MODELS
# =====

comparison = pd.DataFrame({
    "Model": [
        "ANN",
        "ANN + Dropout"
    ],
    "Accuracy": [
        accuracy,
        dropout_accuracy
    ],
    "Loss": [
        loss,
        dropout_loss
    ]
})

print("\n===== MODEL COMPARISON =====")
print(comparison.to_string(index=False))

# =====
# 16. DROPOUT MODEL ACCURACY GRAPH
# =====

plt.figure(figsize=(8, 5))

plt.plot(
    history_dropout.history["accuracy"],
    label="Training Accuracy"
)

plt.plot(
    history_dropout.history["val_accuracy"],
    label="Validation Accuracy"
)

```

```
plt.title("ANN with Dropout - Accuracy")
plt.xlabel("Epoch")
plt.ylabel("Accuracy")
plt.legend()
plt.grid(True)

plt.show()

# =====
# 17. FINAL RESULT
# =====

if dropout_accuracy > accuracy:
    best_model = "ANN + Dropout"
    best_accuracy = dropout_accuracy
else:
    best_model = "ANN"
    best_accuracy = accuracy

print("\n=====")
print("          TASK 6 COMPLETED SUCCESSFULLY")
print("=====")

print("Best Model   :", best_model)
print("Best Accuracy:", round(best_accuracy, 4))

print("\nTopics Completed:")
print("1. Deep Learning")
print("2. Artificial Neural Network (ANN)")
print("3. Input, Hidden and Output Layers")
print("4. ReLU and Sigmoid Activation")
print("5. TensorFlow/Keras")
print("6. Model Training")
print("7. Epochs and Batch Size")
print("8. Loss Function")
print("9. Adam Optimizer")
print("10. Model Evaluation")
print("11. Overfitting Analysis")
print("12. Dropout Regularization")
print("13. Accuracy and Loss Visualization")
print("14. Model Comparison")

print("\nTask 6 Completed!")
```

TensorFlow Version: 2.20.0

===== DATASET =====

Dataset Shape: (569, 30)

Target Shape : (569,)

Number of Features: 30

Classes: [0 1]

===== DATA SPLIT =====

Training Data: (455, 30)

Testing Data : (114, 30)

Feature scaling completed.

===== ANN MODEL =====

/usr/local/lib/python3.13/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input\_shape` to `Input` layer. It will be ignored. Consider passing it to the first layer instead.  
super().\_\_init\_\_(activity\_regularizer=activity\_regularizer, \*\*kwargs)

Model: "sequential"

| Layer (type)    | Output Shape | Param # |
|-----------------|--------------|---------|
| dense (Dense)   | (None, 32)   | 992     |
| dense_1 (Dense) | (None, 16)   | 528     |
| dense_2 (Dense) | (None, 1)    | 17      |

Total params: 1,537 (6.00 KB)

Trainable params: 1,537 (6.00 KB)

Non-trainable params: 0 (0.00 B)

Model compiled successfully.

===== MODEL TRAINING =====

Epoch 1/30

12/12  4s 68ms/step - accuracy: 0.7555 - loss: 0.5602 - val\_accuracy: 0.8901 - val\_loss: 0.4604

Epoch 2/30

12/12  0s 10ms/step - accuracy: 0.9121 - loss: 0.4006 - val\_accuracy: 0.9341 - val\_loss: 0.3450

Epoch 3/30

... ..